UNIVERSIDADE FEDERAL DO TOCANTINS
CAMPUS UNIVERSITÁRIO DE PALMAS
CURSO DE CIÊNCIA DA COMPUTAÇÃO

Laboratório 3 – *Crossbar Switch* e Multiplexadores

Pedro Lucas Moreira

Relatório – Linguagens de Descrição de Hardware – 2026/2 – Outubro de 2026

Professor(a) Orientador(a): Tiago Almeida

The contents of this report are the sole responsibility of the authors.
O conteúdo deste relatório é de única responsabilidade dos autores.

Pedro Lucas Moreira

Laboratório 3 – *Crossbar Switch* e Multiplexadores

Relatório apresentado como parte das atividades da disciplina Linguagens de Descrição de Hardware – 2026/2, Curso de Ciência da Computação, Universidade Federal do Tocantins.

PALMAS / TO

2026

1 QUESTÃO 1

Para que o mesmo código possa ser gravado na placa DE1, as portas usam os nomes da pinagem definida no `compile.tcl`:

Sinal	Porta VHDL	Pino
x_1	`SWITCH_PINS(0)` (SW0)	PIN_L22
x_2	`SWITCH_PINS(1)` (SW1)	PIN_L21
S	`SWITCH_PINS(2)` (SW2)	PIN_M22
y_1	`LEDR0`	PIN_R20
y_2	`LEDR1`	PIN_R19

1.1 Item (a) – WITH, SELECT e WHEN

Cada saída é escolhida por uma atribuição selecionada controlada por S.

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3
4
5  entity crossbar_switch is
6      port (
7          SWITCH_PINS : in  std_logic_vector(2 downto 0);
8          LEDR0       : out std_logic;
9          LEDR1       : out std_logic
10     );
11 end entity crossbar_switch;
12
13 architecture rtl of crossbar_switch is
14 begin
15     with SWITCH_PINS(2) select
16         LEDR0 <= SWITCH_PINS(0) when '0',
17                  SWITCH_PINS(1) when others;
18
19     with SWITCH_PINS(2) select
20         LEDR1 <= SWITCH_PINS(1) when '0',
```

```
21                  SWITCH_PINS(0) when others;
22
23 end architecture rtl;
```

1.2 Item (b) – WHEN ELSE

Mesmo circuito com atribuição condicional: uma linha por saída.

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity crossbar_switch is
5      port (
6          SWITCH_PINS : in  std_logic_vector(2 downto 0);
7          LEDR0       : out std_logic;
8          LEDR1       : out std_logic
9      );
10 end entity crossbar_switch;
11
12 architecture rtl of crossbar_switch is
13 begin
14     LEDR0 <= SWITCH_PINS(0) when SWITCH_PINS(2) = '0' else SWITCH_PINS
           (1);
15     LEDR1 <= SWITCH_PINS(1) when SWITCH_PINS(2) = '0' else SWITCH_PINS
           (0);
16
17 end architecture rtl;
```

1.3 Item (c) – PROCESS

Mesmo circuito dentro de um processo. Como todas as entradas estão na lista de sensibilidade e as duas saídas recebem valor nos dois ramos do `if`, o circuito continua puramente combinacional.

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity crossbar_switch is
5      port (
```

```
        SWITCH_PINS : in  std_logic_vector(2 downto 0);
        LEDR0       : out std_logic;
        LEDR1       : out std_logic
    );
end entity crossbar_switch;

architecture rtl of crossbar_switch is
begin

    begin
        if SWITCH_PINS(2) = '0' then
            LEDR0 <= SWITCH_PINS(0);
            LEDR1 <= SWITCH_PINS(1);
        else
            LEDR0 <= SWITCH_PINS(1);
            LEDR1 <= SWITCH_PINS(0);
        end if;
    end process;

end architecture rtl;
```

1.4 Simulação dos itens (a), (b) e (c)

As três versões têm a mesma *entity* (`crossbar_switch`), então um único testbench serve para todas; os scripts `a.do`, `b.do` e `c.do` mudam apenas o arquivo compilado. O testbench percorre as 8 combinações de entrada, com 10 ns entre cada uma.

```
library ieee;
use ieee.std_logic_1164.all;


architecture sim of tb_crossbar_switch is
    signal sw         : std_logic_vector(2 downto 0); -- S & x2 & x1
    signal y1, y2     : std_logic;
begin
    uut: entity work.crossbar_switch
        port map (
            SWITCH_PINS => sw,
            LEDR0       => y1,
```

```
13              LEDR1       => y2
14          );
15
16      stim: process
17      begin
18          sw <= "000"; wait for 10 ns; -- direto: y1=0 y2=0
19          sw <= "001"; wait for 10 ns; -- direto: y1=1 y2=0
20          sw <= "010"; wait for 10 ns; -- direto: y1=0 y2=1
21          sw <= "011"; wait for 10 ns; -- direto: y1=1 y2=1
22          sw <= "100"; wait for 10 ns; -- cruza : y1=0 y2=0
23          sw <= "101"; wait for 10 ns; -- cruza : y1=0 y2=1
24          sw <= "110"; wait for 10 ns; -- cruza : y1=1 y2=0
25          sw <= "111"; wait for 10 ns; -- cruza : y1=1 y2=1
26          wait;
27      end process;
28
29  end architecture sim;
```

As três simulações (ModelSim, arquivos `a.vcd`, `b.vcd` e `c.vcd`) produziram exatamente o mesmo resultado:

S	x_2	x_1	y_1	y_2	Modo
0	0	0	0	0	direto
0	0	1	1	0	direto
0	1	0	0	1	direto
0	1	1	1	1	direto
1	0	0	0	0	cruzado
1	0	1	0	1	cruzado
1	1	0	1	0	cruzado
1	1	1	1	1	cruzado

1.5 Item (d) – Cascata com `GENERIC` e `GENERATE`

A *entity* `xbar_chain` recebe o número de estágios pelo `generic N`. Os vetores `top` e `bot` representam as linhas de cima e de baixo entre os estágios: a entrada do primeiro estágio é VCC (`'1'`) em cima e GND (`'0'`) embaixo, e o `for generate` instancia N

cópias do `crossbar_switch` do item (a), cada uma controlada por uma chave S_i. A saída y_1 do último estágio vai para o LED.

```
library ieee;
use ieee.std_logic_1164.all;

entity xbar_chain is
    generic (
        N : integer := 5
    );
    port (
        SWITCH_PINS : in  std_logic_vector(N-1 downto 0);
        LEDR0       : out std_logic
    );
end entity xbar_chain;

architecture structural of xbar_chain is
    signal top : std_logic_vector(N downto 0);
    signal bot : std_logic_vector(N downto 0);
begin
    top(0) <= '1'; -- VCC
    bot(0) <= '0'; -- GND

    xbar_gen: for i in 0 to N-1 generate
        xbar_inst: entity work.crossbar_switch
            port map (
                SWITCH_PINS(0) => top(i),          -- x1
                SWITCH_PINS(1) => bot(i),          -- x2
                SWITCH_PINS(2) => SWITCH_PINS(i),  -- S_i
                LEDR0          => top(i + 1),      -- y1
                LEDR1          => bot(i + 1)       -- y2
            );
    end generate xbar_gen;

    LEDR0 <= top(N);

end architecture structural;
```

Cada chave ligada troca as linhas de lugar. Assim, o valor de VCC volta para a linha de cima sempre que um número **par** de chaves estiver ligado, ou seja, LED =

$\overline{S_0 \oplus S_1 \oplus \cdots \oplus S_{N-1}}$. Por isso, mudar qualquer chave sozinha inverte o estado do LED, que é o comportamento do interruptor de corredor citado no enunciado.

O testbench (`tb_xbar_chain.vhdl`, com $N = 5$) confirmou esse comportamento:

$S_4 \ldots S_0$	Chaves ligadas	LED
00000	0	1
00001	1	0
00011	2	1
10101	3	0
11110	4	1
11111	5	0

1.6 Item (e) – `demo_setup` com 5 estágios

O `demo_setup` instancia o `xbar_chain` com `N => 5`, ligando SW0 a SW4 nas chaves S_0 a S_4 e a saída no LEDR0. O projeto (`compile-2.tcl`, projeto `demo_setup_5`) compilou sem erros e ocupou 2 elementos lógicos: o sintetizador reduz toda a cascata de multiplexadores ao XNOR das 5 chaves, como mostra o *netlist* no RTL Viewer.

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity demo_setup is
5      port (
6          SWITCH_PINS : in  std_logic_vector(4 downto 0);
7          LEDR0       : out std_logic
8      );
9  end entity demo_setup;
10
11 architecture rtl of demo_setup is
12 begin
13     chain_inst: entity work.xbar_chain
14         generic map (N => 5)
15         port map (
16             SWITCH_PINS => SWITCH_PINS,
17             LEDR0       => LEDR0
18         );
```

```
19
20 end architecture rtl;
```

1.7 Item (f) – `demo_setup` com 8 estágios

Igual ao item (e), alterando apenas `N => 8` e o tamanho do vetor de chaves (SW0 a SW7).

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3
4
5  entity demo_setup is
6      port (
7          SWITCH_PINS : in  std_logic_vector(7 downto 0);
8          LEDR0       : out std_logic
9      );
10 end entity demo_setup;
11
12 architecture rtl of demo_setup is
13 begin
14     chain_inst: entity work.xbar_chain
15         generic map (N => 8)
16         port map (
17             SWITCH_PINS => SWITCH_PINS,
18             LEDR0       => LEDR0
19         );
20
21 end architecture rtl;
```

2 QUESTÃO 2 – MULTIPLEXADOR COM DECODIFICADOR

2.1 Item (a) – Decodificador 2 para 4

O decodificador recebe a seleção e o sinal de controle. Com En = 0 todas as saídas ficam com 0 e com En = 1 apenas a saida Y, correspondente ao valor as entradas vai para 1

Sinal	Porta VHDL	Pino
w_0	`SWITCH_PINS(0)` (SW0)	PIN_L22
w_1	`SWITCH_PINS(1)` (SW1)	PIN_L21
En	`SWITCH_PINS(2)` (SW2)	PIN_M22
$y_0 \dots y_3$	`LEDR0 ...LEDR3`	PIN_R20, R19, U19, Y19

```vhdl
library ieee;
use ieee.std_logic_1164.all;


entity dec2_4 is
    port (
        SWITCH_PINS : in  std_logic_vector(2 downto 0);
        LEDR0       : out std_logic;
        LEDR1       : out std_logic;
        LEDR2       : out std_logic;
        LEDR3       : out std_logic
    );
end entity dec2_4;

architecture rtl of dec2_4 is
    signal y : std_logic_vector(3 downto 0); -- y3 y2 y1 y0
begin
    --                      En w1 w0
    with SWITCH_PINS select
        y <= "0001" when "100",
             "0010" when "101",
             "0100" when "110",
             "1000" when "111",
```

```
24              "0000" when others; -- En = '0'
25
26      LEDR0 <= y(0);
27      LEDR1 <= y(1);
28      LEDR2 <= y(2);
29      LEDR3 <= y(3);
30
31  end architecture rtl;
```

O testbench (`tb_dec2_4.vhdl`) percorre as 8 combinações de entrada:

En	w_1	w_0	y_3	y_2	y_1	y_0
0	x	x	0	0	0	0
1	0	0	0	0	0	1
1	0	1	0	0	1	0
1	1	0	0	1	0	0
1	1	1	1	0	0	0

2.2 Item (b) – Lógica extra

O circuito da caixa pontilhada da Figura 1 faz o AND de cada entrada de dados w_i com a saída y_i do decodificador e junta os quatro resultados em um OR:

$$f = w_0y_0 + w_1y_1 + w_2y_2 + w_3y_3$$

Como o decodificador ativa apenas um y_i por vez, f é igual à entrada w_i do canal selecionado. Na placa, SW0 a SW3 são w_0 a w_3, SW4 a SW7 são y_0 a y_3 e o LEDR0 mostra f.

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity extra_logic is
5      port (
6          SWITCH_PINS : in  std_logic_vector(7 downto 0);
7          LEDR0       : out std_logic
8      );
9  end entity extra_logic;
```

```
10
11 architecture rtl of extra_logic is
12     signal w : std_logic_vector(3 downto 0);
13     signal y : std_logic_vector(3 downto 0);
14 begin
15     w <= SWITCH_PINS(3 downto 0);
16     y <= SWITCH_PINS(7 downto 4);
17
18     LEDR0 <= (w(0) and y(0)) or
19              (w(1) and y(1)) or
20              (w(2) and y(2)) or
21              (w(3) and y(3));
22
23 end architecture rtl;
```

O testbench ativa um canal de cada vez, testando o w_i do canal em 1 e em 0 enquanto as outras entradas ficam com o valor oposto, para garantir que só o canal ativo influencia a saída:

$y_3 \dots y_0$	$w_3 \dots w_0$	f
0000	1111	0
0001	0001	1
0001	1110	0
0010	0010	1
0010	1101	0
0100	0100	1
0100	1011	0
1000	1000	1
1000	0111	0